

ReactJS - 條件渲染

我們將學習如何根據某些條件在 **React** 中渲染元素/元件。
換句話說，根據一個或多個條件，元件決定它將返回哪些元件。

在 **React** 中，條件渲染的工作方式與條件在 **JavaScript** 中的工作方式相同。我們使用 **JavaScript** 運算子來建立表示當前狀態的元素，然後 **React** 元件更新 UI 以匹配它們。

在 **React** 中進行條件渲染的方法不止一種。讓我們用一個例子來討論所有這些。

1. if-else 方法

此範例根據 `isLoggedIn` 屬性的值呈現不同的問候語。
請考慮以下兩個元件：

```
function UserGreeting(props) {  
  return <h1>Welcome back!</h1>;  
}
```

```
function GuestGreeting(props) {  
  return <h1>Please sign up.</h1>;  
}
```

我們將建立一個問候語元件，該元件根據使用者是否登錄顯示以下任一元件：

```
function Greeting(props) {  
  const isLoggedIn = props.isLoggedIn;  
  if (isLoggedIn) {  
    return <UserGreeting />;  
  }  
  return <GuestGreeting />;  
}
```

```
ReactDOM.render(  
  // Try changing to isLoggedIn={true}:  
  <Greeting isLoggedIn={false} />,  
  document.getElementById('root')  
);  
export default Greeting;
```

2. 元素變數方法

您可以使用變數來儲存元素。這可以說明您有條件地渲染元件的一部分，而輸出的其餘部分不會更改。

請考慮以下兩個表示「登出」和「登入」按鈕的新元件：

```
import React from 'react';
import Greeting from './Greeting'
function LoginButton(props) {
  return (
    <button onClick={props.onClick}>
      Login
    </button>
  );
}
```

```
function LogoutButton(props) {
  return (
    <button onClick={props.onClick}>
      Logout
    </button>
  );
}
```

在下面的範例中，我們將建立一個名為 **LoginControl** 的有狀態元件。它將呈現或取決於其當前狀態：

```
class LoginControl extends React.Component {
  constructor(props) {
    super(props);
    this.handleLoginClick = this.handleLoginClick.bind(this);
    this.handleLogoutClick = this.handleLogoutClick.bind(this);
    this.state = {isLoggedIn: false};
  }

  handleLoginClick() {
    this.setState({isLoggedIn: true});
  }

  handleLogoutClick() {
```

```

    this.setState({isLoggedIn: false});
  }

  render() {
    const isLoggedIn = this.state.isLoggedIn;
    let button;
    if (isLoggedIn) {
      button = <LogoutButton onClick={this.handleLogoutClick} />;
    } else {
      button = <LoginButton onClick={this.handleLoginClick} />;
    }

    return (
      <div>
        <Greeting isLoggedIn={isLoggedIn} />
        {button}
      </div>
    );
  }
}

ReactDOM.render(
  <LoginControl />,
  document.getElementById('root')
);

```

雖然宣告變數和使用 `if` 語句是有條件地呈現元件的好方法，但有時您可能希望使用較短的語法。

讓我們討論幾種在 `React JSX` 中 `inline` 條件的方法。

3. inline If 邏輯 `&&` 運算子

您可以通過將表達式括在大括弧中來嵌入 `JSX` 中的運算式。

這包括 `JavaScript` 邏輯 `&&` 運算子。它對於有條件地包含元素非常方便：

```

import React, { Component } from 'react'

class UserGreeting extends Component {
  constructor(props) {

```

```

    super(props)

    this.state = {
      isLoggedIn: true
    }
  }
  render() {
    return this.state.isLoggedIn && <div>Welcome Ramesh</div>
  }
}
export default UserGreeting

```

它之所以有效是因為在 JavaScript 中，`true && expression` 總是計算為表達式，而 `false && expression` 總是計算為 `false`。

4. 三元算子方法

有條件地 `inline` 呈現元素的另一種方法是使用

JavaScript **條件運算符**條件？ 真：假。

在下面的範例中，我們使用它來有條件地呈現一小塊文本。

```

import React, { Component } from 'react'
class UserGreeting extends Component {
  constructor(props) {
    super(props)
    this.state = {
      isLoggedIn: true
    }
  }
  render() {
    return this.state.isLoggedIn ? (
      <div>Welcome Ramesh</div>
    ) : (
      <div>Welcome Guest</div>
    )
  }
}
export default UserGreeting

```